

Forge - Agent Operating System (Product Specification)

Forge is not an AI code editor. Forge is an Agent Operating System built on top of a VS Code foundation. The goal is to create a local first, agent driven workspace where coding, automation, branding, marketing, research and content creation are handled by a team of specialized AI agents.

Vision

Create a product that feels like a team of AI collaborators rather than a chat interface. The user launches missions. Agents plan, research, execute, review and learn.

Core Principles

1. Local First. 2. Fast and responsive. 3. Multi Agent architecture. 4. Persistent memory and learning. 5. Human approval for sensitive actions. 6. Workspace adapts to the mission. 7. Brand independent from VS Code, Void or Hermes.

Main UI

Mission Bar at the top. Agent Board (Kanban) on the left. Dynamic Workspace in the center. Live Feed on the right. Follow up Chat in a collapsible sidebar. Brain section for memory and learning.

Kanban Agent Board

Columns: Planning Research Working Review Completed Every task moves through the board and shows which agent and model is working on it.

Dynamic Workspace

Coding Mode: File tree, diffs, tests, architecture docs. Automation Mode: Executed actions, browser tasks, filesystem operations, scheduled jobs. Marketing Mode: brand.md, landing.md, seo.md, ads.md, customer_personas.md. Business Mode: Requirements, plans, workflows, reports and assets.

Brain System (Inspired by Hermes)

Project Memory: Architecture, stack, conventions. Skill Memory: Reusable procedures discovered by agents. User Memory: User preferences and working style. Mission Memory: History of completed missions, decisions and outcomes.

Blueprint Documents

Generated before execution: requirements.md architecture.md brand.md design.md landing.md marketing.md seo.md user_personas.md Agents should use these as source of truth.

Agent Team

Architect Researcher Coder QA Reviewer Automation Marketing Designer Each agent may use a different model.

Model Routing

Architect -> Claude / GPT Researcher -> Local model + web search Coder -> Qwen Coder local QA -> Claude / Gemini Automation -> Local model Marketing -> Best available model Users can assign models manually.

Tools

Filesystem access Terminal access Git Docker SSH Browser automation (Playwright) Web Search MCP servers Document generation Image generation PDF generation Spreadsheet generation Email drafting

Web Search Architecture

Web search is a platform capability, not a model capability. Agent -> Search Service -> Sources -> Context -> Model This allows local models to work with fresh information.

Permissions and Security

Level 1: Read operations. Level 2: File modifications require approval. Level 3: Terminal, package installation and Docker actions require approval. Level 4: Delete files, SSH, system changes require explicit approval. Dedicated Security Center UI.

Live Feed

Real time stream of actions: Researching documentation. Running tests. Editing files. Generating assets. Creating automations.

Chat Sidebar

Used for follow ups only. Examples: Change colors. Explain this refactor. Update the brand. Modify the automation. The mission remains the primary interface.

Wow Features

Visible team of agents. Kanban task flow. Persistent Brain. Mission history. Diff viewer. Blueprint generation before coding. Local first experience. Full automation support.

Technology Stack

Frontend: VS Code fork/base React TypeScript Tailwind Backend: Agent Orchestrator Memory Engine Tool Execution Layer Search Layer MCP Integration Models: Ollama OpenRouter OpenAI Anthropic Gemini Custom providers

Development Roadmap

Phase 1: VS Code base. Agent orchestration. Kanban. Memory. Phase 2: Web search. Automation. Blueprint documents. Phase 3: Advanced memory. Marketplace. Team sharing. Phase 4:

Standalone ecosystem.